PORTFOLIO PROJECT PDF

ALY6030 Pharmacy Claims Data Warehousing and SQL Project

This is my shorter portfolio version for ALY6030 - Data Warehousing & SQL in Spring 2025. I used simple CSV prep, MySQL, and SQL reporting logic to turn a small pharmacy claims sample into a clean star schema. My main work was normalization, key setup, ERD design, and reporting queries.

MySQL | SQL | 3NF | Star Schema | ERD

SIMPLE NOTE

This PDF is a cleaner portfolio rewrite from my original course report, SQL script, ERD export, and project files. I kept the story short and focused on the parts I would want to explain in an interview.

BUSINESS PROBLEM

The PBM sample data was not ready for warehouse reporting. I needed to restructure the raw file into a clean dimensional model and prepare example SQL outputs before a larger production rollout.

PROJECT INFO

At a glance

Course: ALY6030 - Data Warehousing & SQL

Term: Spring 2025

Type: Individual project

Team: N/A

Original deliverables: PDF report, SQL script, CSV tables, ERD

Dataset: Pharmacy Claims PBM sample

Rows / columns: Raw 5 x 21 -> modeled 3 tables with 11 fact rows

Main tools: MySQL Workbench, SQL, CSV / spreadsheet editing

Model note: Natural keys in dimensions, surrogate fill_id in fact table

OVERVIEW

Project Snapshot

The original assignment asked me to take a raw pharmacy claims file, normalize it, build a star schema in MySQL, create an ERD, and write business-facing SQL queries. This portfolio version keeps the same full flow, but shows the project in a shorter and more visual way.

BUSINESS FOCUS

Warehouse-ready reporting

Turn a wide raw file into a structure analysts can actually query.

DATA SCALE

5 raw rows -> 11 events

A very small sample, but enough to show the full modeling logic.

METHODS USED

3NF + star schema

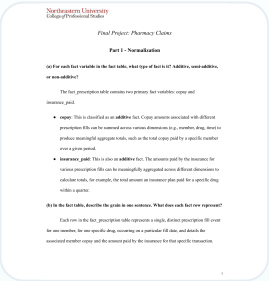
Normalization, PK/FK setup, referential integrity, and SQL reporting.

ORIGINAL DELIVERABLES

PDF + SQL + ERD

Course report, SQL build script, modeled CSV files, and MySQL ERD export.

Original source materials consolidated here



Northeastern University
College of Professional Studies

Final Project: Pharmacy Claims

Part I – Normalization

(a) For each fact variable in the fact table, what type of fact is it? Additive, semi-additive, or non-additive?

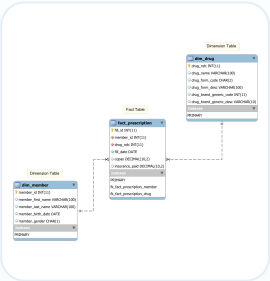
The fact_prescription table contains two primary fact variables: copay and insurance_paid.

- **copay**: This is classified as an **additive** fact. Copay amounts associated with different prescription fills can be summed across various dimensions (e.g., member, drug, time) to produce meaningful aggregate totals, such as the total copay paid by a specific member over a given period.
- **Insurance_paid**: This is also an **additive** fact. The amounts paid by the insurance for various prescription fills can be meaningfully aggregated across different dimensions to calculate totals, for example, the total amount an insurance plan paid for a specific drug within a quarter.

(b) In the fact table, describe the grain in one sentence. What does each fact row represent?

Each row in the fact_prescription table represents a single, distinct prescription fill event for one member, for one specific drug, occurring on a particular fill date, and details the associated member copay and the amount paid by the insurance for that specific transaction.

Original course report



MySQL ERD export

I reused the core logic from my original report and SQL work, then rewrote it into a faster portfolio version. I also kept the course focus aligned with the syllabus: dimensional modeling, SQL, referential integrity, and warehouse thinking.

My role in the project

- Reviewed the raw PBM sample and found why it did not meet 3NF.
- Split the data into dimension and fact CSV tables.
- Built the MySQL schema with primary keys, foreign keys, and referential actions.
- Wrote reporting SQL and summarized the final answers in a clean report.

Contribution note

This was an individual course project, so I handled the full flow myself from data restructuring to SQL reporting and ERD delivery.

BUSINESS PROBLEM AND ANALYSIS PLAN

Problems, Data, and Analysis Plan

This project was mainly a data warehousing task. The raw file mixed member data, drug data, and repeated fill fields in one wide table. My job was to redesign that structure so future reporting analysts and business users could query it more clearly and safely.

PROJECT PURPOSE

I wanted to show that I could read messy business data, identify grain and normalization problems, and rebuild the data into a warehouse-friendly layout. I also wanted to support sample reporting needs before the full PBM feed goes live.

Dataset summary	Details
Main table	final_project_data.csv, a 5 x 21 raw sample with member, drug, and repeated fill fields
Second table	final_project_data_description.csv, a 19-field data dictionary
Merge key	N/A at first - one raw file later split into member, drug, and fill tables
Working data	dim_member (4 x 5), dim_drug (4 x 6), fact_prescription (11 x 5)
Missing values	Mainly in fill_date2/fill_date3 and related payment fields because the raw file used a repeated-group design

STAKEHOLDERS

Reporting analysts, business users, the internal data developer, and the PBM partner that will later send the production claims data.

MAIN QUESTIONS

What should the fact grain be?
How should member and drug data be separated?
How should PK/FK rules protect history?
What example SQL reports will users need first?

MY WORKFLOW

Review raw file -> normalize in CSV/spreadsheet -> split into dimension and fact tables -> build MySQL schema -> set PK/FK -> export ERD -> write reporting SQL.

SELECTED CODE - MAIN SETUP

```
CREATE TABLE fact_prescription (
  fill_id INT AUTO_INCREMENT NOT NULL,
  member_id INT NOT NULL,
  drug_ndc INT NOT NULL,
  fill_date DATE NOT NULL,
  copay DECIMAL(10, 2),
  insurance_paid DECIMAL(10, 2),
  PRIMARY KEY (fill_id)
);

ALTER TABLE fact_prescription
ADD CONSTRAINT fk_fact_prescription_member
  FOREIGN KEY (member_id)
  REFERENCES dim_member(member_id)
  ON DELETE RESTRICT
  ON UPDATE CASCADE,
ADD CONSTRAINT fk_fact_prescription_drug
  FOREIGN KEY (drug_ndc)
  REFERENCES dim_drug(drug_ndc)
  ON DELETE RESTRICT
  ON UPDATE CASCADE;
```

This was the core warehouse setup. I used a surrogate key for each fill event and protected the member and drug links with foreign key rules.

LAB / CODE NOTE

I used spreadsheet-style CSV editing for the normalization step, MySQL Workbench for schema design and ERD export, and one SQL script for database setup plus reporting queries.

EXAMPLE FIELDS I USED

member_id, member_first_name, member_last_name, member_birth_date

drug_ndc, drug_name, drug_form_code, drug_brand_generic_desc

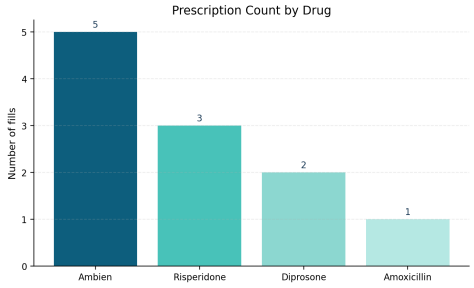
fill_date, copay, insurance_paid, fill_id

EXPLORATORY DATA ANALYSIS

Key Findings and Visual Evidence

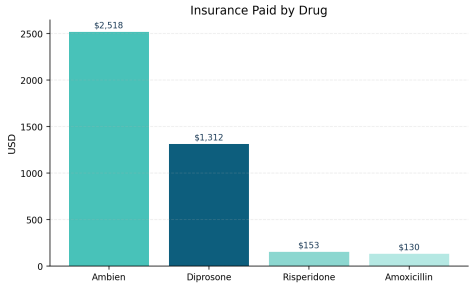
Even though the dataset was very small, it still gave me a clear reporting story. The modeled tables were enough to answer which drugs appeared most often, where insurance spending was highest, and how fills were distributed over time.

Prescription Count by Drug



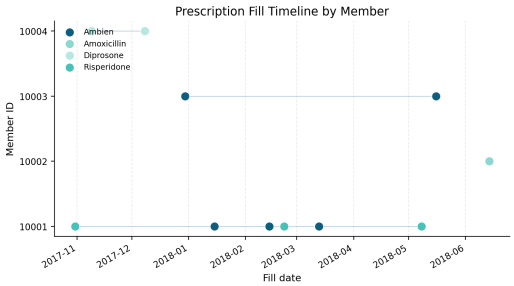
Ambien had the highest fill count in the sample with 5 prescription events, followed by Risperidone with 3.

Insurance Paid by Drug



Ambien also had the highest insurance-paid total at $2,518. Diprosone was second because of two relatively expensive fills.

Prescription Fill Timeline by Member



The timeline shows repeated fills across late 2017 to mid 2018. One older member drove more repeated activity than the rest of the sample.

WHAT THESE CHARTS TOLD ME

- Ambien was the top drug by both fill count and insurance-paid total.
- The final model expanded 5 raw rows into 11 true prescription fill events.
- The age 65+ group had 1 unique member and 6 total fills in this sample.
- The latest-fill query showed that member 10003 most recently filled Ambien with $322 paid by insurance.

Quick business read

Even a small warehouse prototype can give business users fast answers once the data grain is fixed and the schema is clean.

METHODS OR MODELING NOTE

Methods, Relationship Checks, or Model Note

I did not build a predictive model here. The main technical value was data modeling logic and SQL design. The most useful reporting step was ranking the latest fill per member with a window function.

SELECTED CODE SNIPPET

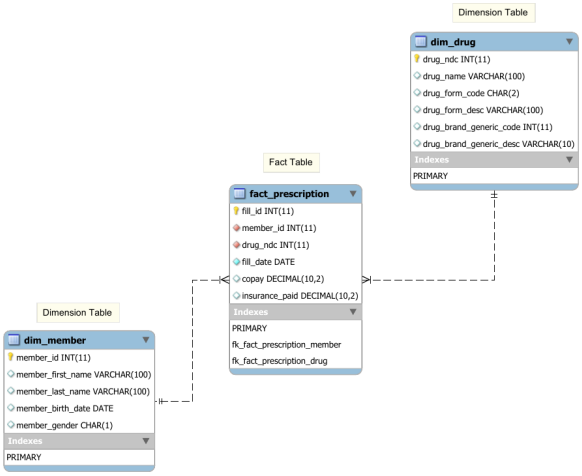
```
WITH RankedFills AS (
  SELECT
    f.member_id,
    m.member_first_name,
    m.member_last_name,
    d.drug_name,
    f.fill_date,
    f.insurance_paid,
    ROW_NUMBER() OVER (
      PARTITION BY f.member_id
      ORDER BY f.fill_date DESC
    ) AS rn
  FROM fact_prescription f
  JOIN dim_member m ON f.member_id = m.member_id
  JOIN dim_drug d ON f.drug_ndc = d.drug_ndc
)
SELECT member_id, drug_name, fill_date, insurance_paid
FROM RankedFills
WHERE rn = 1;
```

This query let me answer the assignment question about each member's most recent prescription fill in a clean and reusable way.

INTERPRETATION

- I kept business identifiers as natural keys in the member and drug dimensions.
- I used a surrogate fill_id so each prescription event had one clean row identifier.
- ON DELETE RESTRICT protected historical claims context from accidental dimension deletion.
- ON UPDATE CASCADE kept the fact-to-dimension relationships aligned if a key value changed.

Star Schema ERD



This simple ERD shows the final three-table design: fact_prescription in the middle, with member and drug dimensions around it. I kept dates in the fact table because the assignment did not require a separate date dimension.

EXECUTIVE SUMMARY

Conclusion, Recommendations, and Interview Use

The big value of this project was not data size. It was the full process: reading a messy raw file, deciding the right grain, building a small but valid warehouse model, and turning it into business-ready SQL outputs.

MAIN FINDINGS

The raw file had repeated fill columns and duplicated member context, so it was not in good warehouse form.

The final model created 11 fact rows and supported direct reporting by drug, age group, and latest fill event.

RESULT MEANING

This prototype shows how a warehouse design can make pharmacy claims easier to query before a larger PBM production feed arrives.

RECOMMENDATIONS

Add a date dimension for production scale, replace manual inserts with a repeatable load pipeline, and keep the data dictionary aligned with the raw field names.

WHAT THIS PROJECT SHOWS ABOUT MY WORK

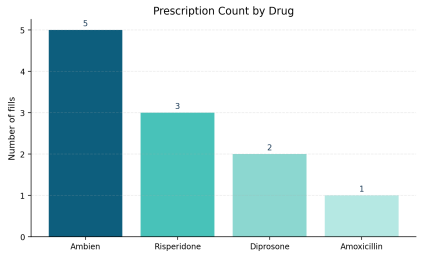
- I can read messy source data and identify structural problems.
- I can build clear fact and dimension tables with practical key choices.
- I can write SQL with joins, aggregation, CASE logic, and window functions.
- I can explain technical work in a simple business-friendly way.

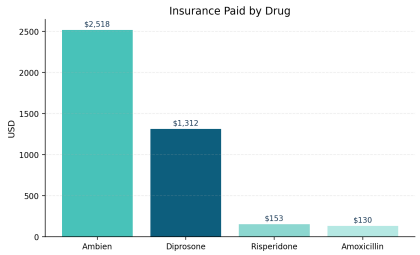
HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

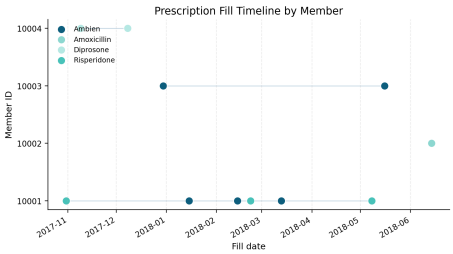
Simple version: In this project I took a small pharmacy claims sample that was not in good warehouse shape. I normalized it into member, drug, and prescription tables, set keys in MySQL, built an ERD, and wrote sample SQL reports. It was a small dataset, but it showed that I can turn raw business data into a structure analysts can actually use.

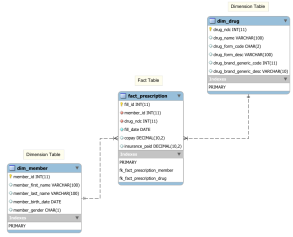
> **Final note**
> This was a course-scale prototype, so the main value is the design logic, SQL workflow, and communication, not a large production-sized dataset.

Quick Visual Gallery









I kept a small gallery at the end so an interviewer can scan the full project story in a few seconds: top pattern, cost pattern, event timeline, and final schema.